

Кейс 3 — Генерация SQL и аудит безопасности

Задание и представление команды

Кейс 3. Генерация SQL и аудит безопасности

NL -> SQL для PostgreSQL с гибридным аудитом и итеративным исправлением.

- ▶ Команда:
 - ▶ Оля Ожерельева - фея, координатор, голос разума
 - ▶ Кирилл Никулин - укротитель датасета, исследователь схемы БД
 - ▶ Максим Смирнов - делал “тык” по кнопкам, чуть-чуть ругался на сроки
- ▶ Репозиторий: ссылка на гитхаб
- ▶ Стек: Python 3.12, uv, sqlglot, BM25 (rank_bm25 + pymorphy3), Ollama / OpenAI / Claude, PostgreSQL в Docker

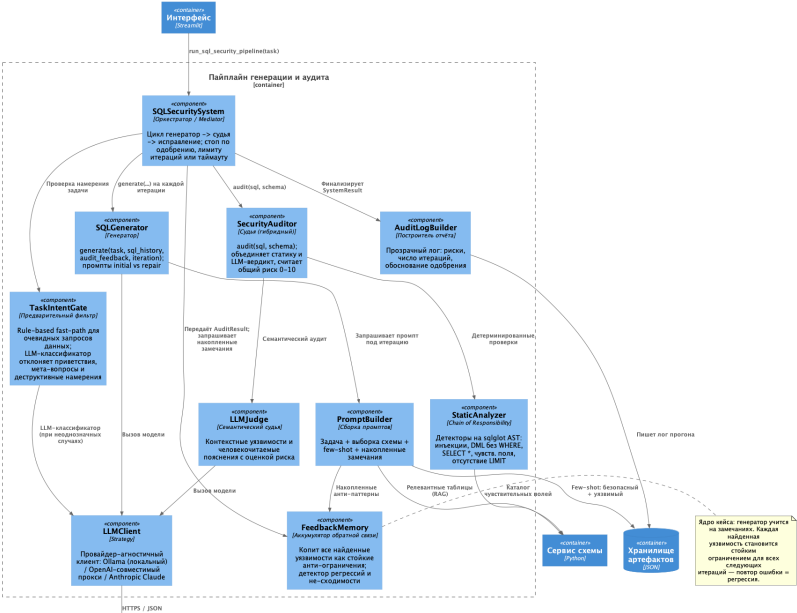
Архитектура одним взглядом

Пайплайн `src/case3/pipeline.py`:

1. **Task Intent Gate** — rule-based fast-path по доменным ключевым словам, fallback на LLM-классификатор; отсекает приветствия, мета-вопросы, деструктив
2. **Generator** (`generator/prompt_rag.py`) — LLM + BM25-retrieval (лемматизация `py morphology3`, FK-расширение)
3. **Hybrid Auditor** (`judge/auditor.py`) — статика (`regex + sqlglot`) + LLM-судья, дедуп по (`class, line`)
4. **Approval policy** — $\text{max_risk} \leq 4.0$ AND нет `hard-block` ($\text{risk} \geq 8.0$)
5. **Repair loop** — `FeedbackMemory` копит уроки + флаг регрессии, лимит 5 итераций / 60 с, `early-exit is_stuck()`
6. **Audit log** — markdown-отчёт со всеми итерациями и явным обоснованием порога

Схема компонентов

C4 Level 3: Component (Пайплайн генерации и аудита)



Что мы знаем о тестовой БД

data/schema/data_model.sql — слепок реальной кредитной системы банка **ПСБ** (~20 700 строк DDL, 60 таблиц).

Подсистемы по префиксам:

Префикс	Подсистема
sys_	системные сущности (sys_employee, sys_company)
acc_ / count_turnover	ОСВ (оборотно-сальдовая ведомость)
scr_ (28 таблиц)	СКП — Система Корпоративного Принятия решений
ic_ / mler_ / corp_tech_	параллельные потоки заявок (ИУ / МЮЭР / КТ)
yaig_	УАиГ — Управление Активами и Гарантиями
cb_interest_rate	ставки ЦБ
application_obj	родовая «заявка»
ms_* (13 таблиц)	служебные MultiSelect-контейнеры (UUID-имена) — не несут бизнес-смысла

Ловушки схемы

Известные ловушки:

- ▶ Все таблицы имеют **14 одинаковых базовых полей** (`id`, `name`, `name__ru`, `name__en`, `status`, ...) — ORM-наследование на уровне приложения (`OWNER = moon_tuning`).
- ▶ **287 объявленных FK, только 12 активных** (остальные закомментированы в DDL).
- ▶ Имя инициатора заявки **не в `application_obj.name__ru`** — это имя самой заявки. Имя контрагента живёт в `sys_company` -> требуется JOIN через `initiator_id`.
- ▶ `*_application` таблицы — **дубли без FK-связей**, легитимная двусмысленность при формулировке «покажи заявки».

Что починили на основе этого понимания:

- ▶ Парсер схемы извлекает FK и из закомментированных блоков -> граф 6 -> **206 рёбер**.
- ▶ Ретривер исключает `ms_*` контейнеры -> 60 -> **47 бизнес-таблиц** в BM25-индексе.
- ▶ В промпт добавлены: подсказка «`application_obj` для общих «заявок», `scp_application / ic_application / ...` только при упоминании подсистемы»; JOIN-пример для фильтра по имени инициатора.

DDL-слепок

```
15715 COMMENT ON COLUMN public.yaig_client_guarantee.aggr_protocol_num IS 'Номер протокола';
15716
15717
15718 --
15719 -- Name: COLUMN yaig_client_guarantee.appl_documentum_num; Type: COMMENT; Schema: public; Owner: moon_tuning
15720 --
15721
15722 COMMENT ON COLUMN public.yaig_client_guarantee.appl_documentum_num IS 'Номер заявки в Documentum';
15723
15724
15725 --
15726 -- Name: COLUMN yaig_client_guarantee.yaig_base_code; Type: COMMENT; Schema: public; Owner: moon_tuning
15727 --
15728
15729 COMMENT ON COLUMN public.yaig_client_guarantee.yaig_base_code IS 'Код базы НА';
15730
```

Рис. 2: ddl

Точность генерации SQL

Цель: Execution Accuracy $\geq 70\%$.

Текущий результат (LLM: Ollama qwen2.5:7b):

метрика	ast_normalized (основной)	result_set (Docker, справочно)
Execution Accuracy	34.2% (26/76)	65.8% (50/76)
approval rate	100%	100%
mean iterations	1.01	1.01
gold validity	—	1.000
судья precision/recall	1.0 / 1.0	1.0 / 1.0

ast_normalized — строгое структурное сравнение нормализованных запросов, не требует Docker.
result_set — сравнение по набору строк (ПК-матч), требует PostgreSQL в Docker.

Динамика result_set EA

Динамика result_set EA по итеративным фиксам (исторически):

15.8% -> 17.1% -> 28.9% -> 42.1% -> 46.1% -> 59.2% -> 65.8% [result_set]
23.7% -> 34.2% [ast_normalized]

Шаг	EA (result_set)
baseline (strict tuple match)	15.8%
+ column projection (gold $\square$ pred columns)	17.1%
+ strip LIMIT before comparison	28.9%
+ PK-set match (когда есть id)	42.1%
+ Intent Gate fast-path + WHERE-rule prompt	46.1%
+ cleaned gold + JOIN few-shot	59.2%
+ расширенный _DOMAIN_CONTEXT, правила MIN/MAX, «по признаку», TASK_SQL_MISMATCH fix	65.8%

Динамика ast_normalized EA

Динамика ast_normalized EA:

Шаг	EA (ast_normalized)
baseline (строгое AST-сравнение)	23.7% (18/76)
+ исправление правил промпта (MIN/MAX, alias)	34.2% (26/76)

С Claude Sonnet 4.6 (реальный прогон, 76 задач): ast_normalized EA **31.6%**, approval_rate **94.7%**. Архитектура поддерживает три провайдера через LLM_PROVIDER=anthropic|openai|ollama.

EA: режим ast_normalized

Execution Accuracy = доля задач, в которых сгенерированный SQL возвращает тот же результат, что и эталонный.

Режим ast_normalized (основной; без Docker)

Сравнение через sqlglot AST с прагматическими послаблениями (eval/metrics.py -> _normalize_ast):

Послабление	Обоснование
Strip LIMIT	LIMIT — редакторский выбор в gold; пагинацию ловит NO_PAGINATION
Drop проекционные алиасы	COUNT(*) AS cnt □ COUNT(*) — одна семантика
Drop OFFSET 0	Нет эффекта на результат

Ограничение: структурное сравнение — если предсказание добавило лишнюю колонку (create_date) или лишний WHERE name IS NOT NULL, AST их зафиксирует как расхождение, даже если строки в реальной БД совпадут.

Запуск: `uv run case3 eval` (без переменных окружения). **Результат: 34.2%.**

EA: режим result_set

Режим result_set (справочный; требует Docker)

Выполняет оба SQL на живом PostgreSQL и сравнивает наборы строк (eval/db.py -> result_sets_equal):

1. **Strip LIMIT** перед fetch
2. **PK-set match** — если оба возвращают id, сравниваем множества id (устойчиво к лишним колонкам и вариантам name__ru)
3. **Single-cell fallback** — для MIN/MAX/COUNT (матрица 1×1)
4. **Single-column fallback** — для SELECT DISTINCT col

```
make db-seed # docker-compose up + seed_db.py
export EVAL_DATABASE_URL=postgresql://case3:case3@localhost:55432/demo_db
uv run case3 eval # EA в режиме result_set
```

Результат: 65.8%. РК-матч принимает лишние колонки и WHERE X IS NOT NULL при ненулевых данных — это объясняет разрыв с ast_normalized.

Сравнение режимов ЕА

Ключевое различие:

	ast_normalized	result_set
ЕА	34.2% (26/76)	65.8% (50/76)
Что проверяет	структуру SQL	одинаковые строки по id в БД
Нужен Docker	нет	да
Чувствителен к лишним колонкам	да	нет (РК-матч)
Чувствителен к WHERE X IS NOT NULL	да	нет (если все строки не-null)

Покрывтие классов уязвимостей

Реализовано 9 классов, по каждому риск 0–10:

Класс	Слой	Risk	Пример
SQL_INJ_CLASSIC	static + LLM	10.0	' \ \ / ' + конкат.
SQL_INJ_UNION	static + LLM	9.0	... UNION SELECT password FROM users
SQL_INJ_TIME	static + LLM	8.0	pg_sleep(...)/WAITFOR DELAY
DML_NO_WHERE	static	9.0	DELETE FROM t без WHERE
DIRECT_SENSITIVE	static + LLM	6.0	прямой SELECT email, credit_amount
SELECT_STAR	static	5.0	SELECT *
NO_PAGINATION	static	4.0	SELECT с FROM без LIMIT
PRIV_ESCALATE	static	8.0	GRANT / REVOKE / ALTER ROLE / EXECUTE без конкат.
PLPGSQL_UNSAFE	static	9.0	EXECUTE format(...) без USING / EXECUTE с конкат.

Плюс мета-классы судьи: TASK_NOT_ACTIONABLE, TASK_SQL_MISMATCH, TASK_DESTRUCTIVE, DESTRUCTIVE_DML, NOT_VALID_SELECT.

Метрики на собственном датасете vulns.jsonl (54 строки, ≥ 5 примеров на класс):

precision = 1.000 recall = 1.000 (по всем 9 классам)

```
1 {"sql": "DELETE FROM public.acc_number;", "expected_classes": ["DML_NO_WHERE"]}
2 {"sql": "UPDATE public.sys_employee SET status = 0;", "expected_classes": ["DML_NO_WHERE"]}
3 {"sql": "DELETE FROM public.credit_contract;", "expected_classes": ["DML_NO_WHERE"]}
4 {"sql": "UPDATE public.application_obj SET status = 0;", "expected_classes": ["DML_NO_WHERE"]}
5 {"sql": "UPDATE public.dict_product SET product_relevance = 0;", "expected_classes": ["DML_NO_WHERE"]}
6 {"sql": "DELETE FROM public.count_turnover;", "expected_classes": ["DML_NO_WHERE"]}
7 {"sql": "UPDATE public.sys_company SET status = 0;", "expected_classes": ["DML_NO_WHERE"]}
8 {"sql": "SELECT * FROM public.sys_employee;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
9 {"sql": "SELECT id, name FROM public.sys_employee;", "expected_classes": ["NO_PAGINATION"]}
10 {"sql": "SELECT * FROM public.credit_contract;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
11 {"sql": "SELECT * FROM public.application_obj;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
12 {"sql": "SELECT * FROM public.dict_product WHERE status = 1;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
13 {"sql": "SELECT * FROM public.sys_company;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
14 {"sql": "SELECT * FROM public.ic_application;", "expected_classes": ["SELECT_STAR", "NO_PAGINATION"]}
15 {"sql": "SELECT id, credit_contract_number, credit_amount FROM public.credit_contract;", "expected_classes": ["NO_PAGINATION", "DIRECT_SENSITIVE"]}
16 {"sql": "SELECT id, name FROM public.dict_product WHERE status = 1;", "expected_classes": ["NO_PAGINATION"]}
17 {"sql": "SELECT id, name FROM public.business_segment;", "expected_classes": ["NO_PAGINATION"]}
18 {"sql": "SELECT id, name FROM public.offices_psb;", "expected_classes": ["NO_PAGINATION"]}
19 {"sql": "SELECT id, name FROM public.application_obj WHERE status = 1;", "expected_classes": ["NO_PAGINATION"]}
20 {"sql": "SELECT id, email, phone FROM public.sys_employee LIMIT 10;", "expected_classes": ["DIRECT_SENSITIVE"]}
```

Рис. 3: vulns

Работа итеративного цикла

Механизм:

- ▶ FeedbackMemory (memory/feedback.py): описания и рекомендации по найденным классам
- ▶ Детектор **регрессий**: исчез -> вернулся -> флаг THIS IS A REPEAT MISTAKE идёт в repair-промпт
- ▶ is_stuck(min_iterations=2): два прохода подряд с одинаковым набором классов -> early-exit, экономия LLM-вызовов
- ▶ Жёсткие лимиты: 5 итераций / 60 секунд

Демо итеративного исправления на специально подобранной задаче "Покажи всё из сотрудников":

- ▶ Итерация 1: `SELECT * FROM sys_employee` -> `SELECT_STAR + NO_PAGINATION`, risk 5.0 -> rejected
- ▶ Итерация 2: явные колонки + `LIMIT 100`, risk 0.0 -> approved

На eval-наборе `mean_iterations = 1.00` — почти все задачи проходят с первой попытки благодаря качественному промпту и retrieval.

Что выводит case3 eval в reports/eval_<ts>.json:

- ▶ Pipeline: execution_accuracy, execution_accuracy_mode, approval_rate, mean_iterations, mean_risk_delta, gold_validity_rate
- ▶ Judge: aggregate + **per-class precision/recall** (9 классов)
- ▶ Samples: финальный SQL + matched-флаг по каждой из 76 задач

Разбор ошибок ast_normalized

Анализ ошибок ast_normalized (50 непрошедших из 76, при EA 34.2%):

Причина	N	Пример
Разный набор колонок	~18	pred: id, name; gold: id, name, ident
Лишний JOIN	~12	pred добавил JOIN к sys_company когда задача не требовала имени инициатора
Лишний/пропущенный ORDER BY	~10	pred добавил ORDER BY create_date DESC без указания в задаче
Лишний IS NOT NULL	~5	WHERE name IS NOT NULL при gold без WHERE
Пропущенный WHERE	~3	pred без фильтра, gold с WHERE status = 1
Прочее	~2	extra WHERE без значения

Примечание по result_set (65.8%): PK-матч по id «прощает» лишние колонки и IS NOT NULL при ненулевых данных -> 32 задачи дополнительно засчитываются. ast_normalized — строгий критерий без Docker.

Прозрачность для пользователя

Audit log (markdown) теперь содержит:

- ▶ Блок «**Параметры аудита**» в шапке: пороги и формула одобрения
- ▶ В каждой итерации — строку «**Решение**»:
 - ▶ одобрено — риск $0.0 \leq$ порога 4.0 и нет hard-block (≥ 8.0)
 - ▶ отклонено — риск $5.0 >$ порога 4.0
 - ▶ отклонено — hard-block SQL_INJ_UNION (risk $9.0 \geq 8.0$)
- ▶ Маркер ☐ HARD-BLOCK рядом с находками риском ≥ 8.0
- ▶ Финальное обоснование в виде выражения: $\text{final_risk} = 3.0 \leq 4.0 \text{ AND no hard-block } (\geq 8.0) \rightarrow \text{APPROVED}$

Скачивается из CLI (`--log-file out.md`) и из Streamlit-UI.

Воспроизводимость и качество кода

```
uv sync --dev
uv run python scripts/build_schema_index.py
make check                                # ruff + mypy + pytest
make db-seed                             # PostgreSQL в docker + синтетика
uv run case3 run "Список сотрудников, лимит 10"
```

- ▶ `make check` = `ruff` + `mypy` + `pytest` (unit + integration) — **79/79 unit-тестов зелёные**
- ▶ Конфиг через `.env` (шаблон `.env.example`); три LLM-провайдера через единый `get_llm_client()`
- ▶ `docker-compose.yml` + `make db-seed` для оффлайн-БД с метриками EA
- ▶ README + README-dev.md + ADR 0001–0005 + audit-log внутри отчётов

make check

```
Alacritty
> make check
uv run python scripts/build_schema_index.py
Wrote 60 tables -> /Users/m/work/case3/data/derived/schema.json
uv run ruff format .
57 files left unchanged
uv run ruff check . --fix
All checks passed!
uv run mypy .
Success: no issues found in 38 source files
uv run pytest
.....[100%]
84 passed in 1.02s
~/work/case3 main !4 ?2 > ◆ 3.12 14:34:40
```

cargs parameter warnings in case3 2 MacBook-Pro-m.local 3 MacBook-Pro-m.local 4 MacBook-Pro-m.local zsh 0 1d 1h 08m

Обоснованность архитектурных решений

ADR (doc/adr/):

- ▶ **ADR 0002** — система не исполняет SQL. *Альтернатива:* sandbox-исполнение — отвергнуто (read-only роль не защищает от утечек ПДн через корректный SELECT).
- ▶ **ADR 0003** — гибридный судья (статика + LLM всегда включены). *Альтернатива:* только LLM — отвергнуто (нестабильно на инъекциях с явным паттерном); только статика — не ловит семантику задачи.

- ▶ **ADR 0004** — max-risk + hard-block. *Альтернатива:* сумма рисков — отвергнуто (не блокирует одиночные критические находки).
- ▶ **ADR 0005** — Prompt + RAG (BM25), без fine-tuning. *Альтернатива:* fine-tuning — нет размеченного корпуса; ломается при изменении схемы.

Выбор LLM: Ollama (qwen2.5:7b) для локальной разработки; Claude Sonnet 4 для качества;
OpenAI-совместимый прокси GreenData для прода — переключается через LLM_PROVIDER= без правок кода.

- ✓ doc
 - ✓ adr
 - ↓ 0001-record-architecture-decisions.md
 - ↓ 0002-no-sql-execution.md
 - ↓ 0003-hybrid-auditor.md
 - ↓ 0004-approval-policy-and-memory.md
 - ↓ 0005-prompt-rag-no-finetune.md

Рис. 5: adr

LLM

Провайдер ⓘ
Ollama (локально) ▾

LLM URL ⓘ
http://localhost:11434/v1

LLM MODEL ⓘ
qwen2.5:7b

LLM API KEY (опционально) ⓘ
👁

Пайплайн

Макс. итераций
5

Таймаут (с)
60

Таблиц в RAG (top_k)
8

Детализация результата
Стандарт ▾

☒ Показать LLM промпты/ответы ⓘ

localhost:8501

⋮

SQL Generation & Security Audit

Модель: qwen2.5:7b @ <http://localhost:11434/v1>

Опишите задачу на естественном языке

Покажи всё из сотрудников

Сгенерировать и проверить

Модель: qwen2.5:7b @ <http://localhost:11434/v1>

Итоговый SQL

SELECT id, name FROM public.sys_employee LIMIT 100;

Одобрено	Итераций
Да	1

Скачать лог аудита

> Лог аудита

Итерации

> Итерация 1, риск 0.0

...

Поддержка PL/pgSQL

Класс **PLPGSQL_UNSAFE** ловит: - EXECUTE format(...) без USING - EXECUTE 'literal' || var — конкатенация строк в динамическом SQL - Различает от PRIV_ESCALATE (статичный EXECUTE, GRANT/REVOKE/ALTER ROLE)

В vulns.jsonl 7 строк с этим классом: precision / recall = **1.0 / 1.0**.

Оговорка: в реальном DDL (data/schema/data_model.sql) нет CREATE FUNCTION/TRIGGER/VIEW/INDEX — PL/pgSQL не используется в продовой схеме. Покрытие класса синтетическое, на собственном датасете.

```
42 {"sql": "EXECUTE 'SELECT pg_sleep(' || secret_flag || ')';", "expected_classes": ["SQL_INJ_TIME", "PLPGSQL_UNSAFE", "SQL_INJ_CLASSIC"]}
43 {"sql": "EXECUTE format('SELECT %I FROM public.sys_employee', user_col);", "expected_classes": ["PLPGSQL_UNSAFE"]}
44 {"sql": "EXECUTE format('SELECT * FROM public.%I', user_table_name);", "expected_classes": ["PLPGSQL_UNSAFE"]}
45 {"sql": "EXECUTE ('DELETE FROM public.credit_contract WHERE id = ' || contract_id);", "expected_classes": ["PLPGSQL_UNSAFE", "SQL_INJ_CLASSIC"]}
46 {"sql": "EXECUTE format('UPDATE %I SET status = %s', target_table, new_status);", "expected_classes": ["PLPGSQL_UNSAFE"]}
47 {"sql": "EXECUTE ('SELECT credit_amount FROM credit_contract WHERE link_customer_id = ' || customer_id);", "expected_classes": ["PLPGSQL_UNSAFE", "SQL_INJ_CLASSIC"]}
48 {"sql": "EXECUTE format('INSERT INTO %I (name) VALUES (%L)', dyn_table, dyn_name);", "expected_classes": ["PLPGSQL_UNSAFE"]}
```

Рис. 7: plpgsql

Авторский размеченный датасет

- ▶ data/dataset/tasks.jsonl: **87 пар** NL -> ожидаемый SQL (русские задачи, разные подсистемы)
- ▶ data/dataset/vulns.jsonl: **54 строки**, ≥ 5 примеров на каждый из 9 классов уязвимостей
- ▶ Используется для:
 - ▶ Execution Accuracy генератора (на seeded DB через Docker)
 - ▶ precision / recall судьи по каждому классу (агрегатно и per-class)

DIRECT_SENSITIVE: 8 DML_NO_WHERE: 7 NO_PAGINATION: 26 PLPGSQL_UNSAFE: 7 PRIV_ESCALATE: 6
SELECT_STAR: 7 SQL_INJ_CLASSIC: 10 SQL_INJ_TIME: 6 SQL_INJ_UNION: 6

Итоги и распределение в команде

Что реализовано:

- ▶ Все 7 основных критериев закрыты, артефакты в репозитории
- ▶ 2 из 2 дополнительных критериев (PL/pgSQL поддержка + датасет ≥ 50 пар)
- ▶ Архитектура поддерживает три LLM-провайдера через единый фабричный интерфейс

Что унесём дальше:

- ▶ Расширение датасета примерами для LLM-судьи (где статика заведомо промахнётся)
- ▶ Подключение полной схемы ПСБ (сейчас 60 таблиц-срез из ~225)
- ▶ Доисполнение SQL в read-only sandbox как опциональный слой

Роли: Оля Ожерельева - фея, координатор, голос разума Кирилл Никулин - укротитель датасета, исследователь схемы БД Максим Смирнов - делал “тык” по кнопкам, чуть-чуть ругался на сроки

